Spectral Filtering for Vision-Language-Action Models

Team: Shivam Kak, Ishan Saha, Connor Brown

Abstract

Vision-language-action (VLA) models such as $\pi_{0.5}$ predict action chunks via iterative denoising, but prediction quality can degrade across the temporal horizon due to limited long-range dependency modeling. Spectral Transfer Units (STUs) leverage Hankel matrix eigendecomposition for efficient spectral filtering and have shown strong performance in capturing long-horizon dependencies. We integrate STU layers into the action prediction head of $\pi_{0.5}$ within the `openpi` framework, inserting a residual STU block between the transformer output and action projection. On the LIBERO manipulation benchmark, STU-K4 converges to within 7% of baseline training loss while adding only 2.8% more parameters and no wall-clock overhead.

Background

VLA Models and $\pi_{0.5}$. The π_0 family [1] combines a SigLIP vision encoder and PaliGemma language model with a lightweight action expert. In the flow-matching variant ($\pi_{0.5}$), actions are predicted by denoising a noise vector over multiple steps, producing a velocity field v_t that guides the ODE $x_{t+dt} = x_t + dt \cdot v_t$. While parallel prediction avoids compounding errors of autoregressive decoding, the transformer's causal attention limits explicit temporal modeling across the action horizon.

Spectral Transfer Units. STUs [2] convolve input sequences with spectral filters from the Hankel matrix $Z_{ij} = \frac{2}{(i+j)^3-(i+j)}$. The top-$K$ eigenvectors ϕ_k, scaled by the fourth root of their eigenvalues, serve as filters:

$$y = \sum_{k=1}^{K} \left((u * \phi_k^+) \cdot M_k^+ + (u * \phi_k^-) \cdot M_k^- \right) \tag{1}$$

where $M_k^{\pm} \in \mathbb{R}^{D_{\text{in}} \times D_{\text{out}}}$ are learnable projections and convolution uses FFT with even/odd decomposition.

Method

Architecture. We insert a residual STU block after the transformer action expert. Given hidden states $h \in \mathbb{R}^{B \times H \times W}$ (batch, action horizon, width), we apply $h' = h + \text{STU}(\text{LayerNorm}(h))$ before the final projection $a = W_{\text{out}} \cdot h'$ to action space. The STU operates over the action horizon ($L = H$), treating each position's W-dimensional feature as the input signal. The residual connection ensures the model can fall back to baseline behavior during early training when STU parameters are randomly initialized.

Implementation. We implement `PI0STUPytorch` extending `PI0Pytorch`, overriding `forward` and `denoise_step` to insert the STU block. Spectral filters are precomputed at initialization and registered as non-persistent buffers. Learnable parameters are $M_k^+, M_k^- \in \mathbb{R}^{K \times W \times W}$, initialized at scale $(KW)^{-0.5}$. For the Gemma 300M action expert ($W = 1024$), STU-K4 adds $2 \times 4 \times 1024^2 \approx 8.4$M parameters (2.8% overhead).

Training. All models fine-tune from the pre-trained $\pi_{0.5}$ checkpoint on LIBERO: AdamW (lr $= 5 \times 10^{-5}$), cosine decay with 1000-step warmup, batch size 8, gradient clipping at 1.0, 5000 steps. STU weights (loaded with `strict=False`) train jointly. Action horizon $H = 10$, action dim $D_a = 32$. Training uses bfloat16 on a single A100 80GB.

Experiments

We compare on LIBERO: (1) **Baseline** ($\pi_{0.5}$, no STU), (2) **STU-K4** (K=4 spectral filters, +8.4M params), (3) **STU-K8** (K=8, +16.8M params). We report training loss (MSE between predicted and target velocity fields). Figure 1 shows smoothed loss curves; Table 1 gives per-interval averages.

Model	Final Loss	Init Loss	Extra Params	s/step
Baseline	**0.029**	0.070	0	1.53
STU-K4	0.031	0.141	8.4M (2.8%)	1.48
STU-K8	0.031	0.101	16.8M (5.6%)	1.54

Table 1: Summary of training results. Final loss is averaged over the last 100 steps; init loss is the average over steps 0–999. Both STU variants converge to within 7% of the baseline.

Discussion

Both STU variants converge to a final loss of 0.031, within 7% of the baseline (0.029), despite millions of randomly initialized parameters. The convergence trajectory is notable: STU-K4 starts at 2× higher loss while STU-K8 starts at 1.4× higher—suggesting more spectral filters provide a broader basis that adapts faster. Both catch up by step 2000. This confirms the residual connection works as intended: the model initially bypasses the STU and gradually incorporates

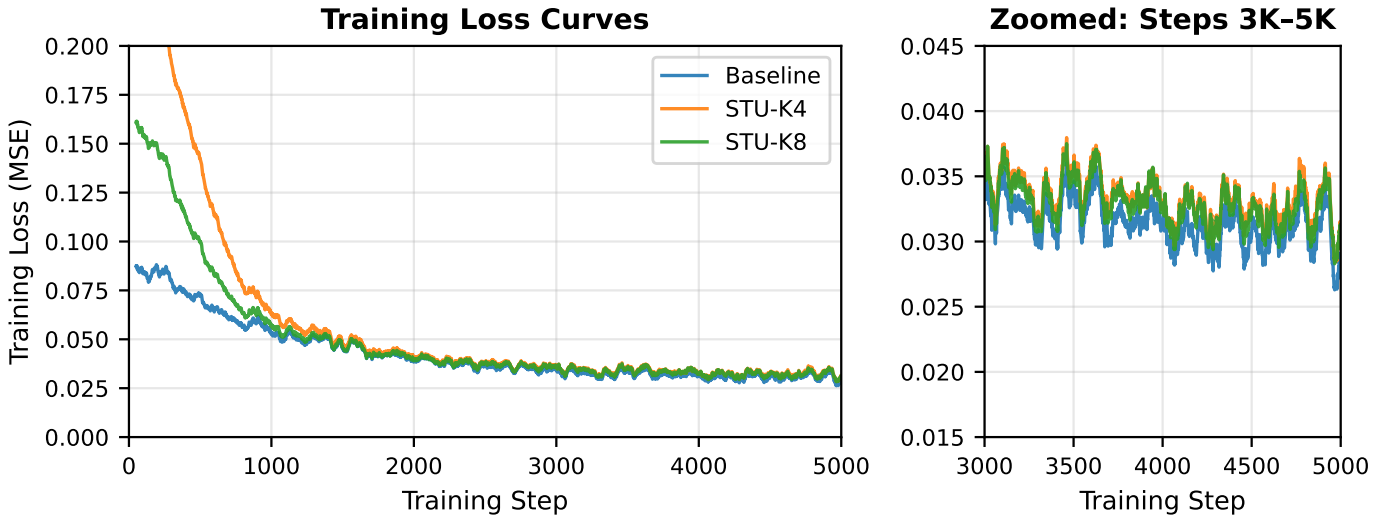

Figure 1: Training loss curves (50-step rolling average). Left: full 5000-step training. STU variants start with higher loss due to random initialization of spectral projection matrices but converge to overlap with the baseline by step ∼2000. Right: zoomed view of steps 3000–5000 showing all three models are nearly indistinguishable.

spectral filtering. Interestingly, K4 and K8 converge to identical final loss despite K8's 2× larger parameter budget, suggesting the short horizon (H=10) saturates the benefit of additional filters.

Computational overhead is minimal (∼1.5 s/step for all models). A key limitation is LIBERO's short action horizon, providing few timesteps for spectral filtering. Future work should evaluate on longer horizons (H=50+) and the autoregressive π_0-FAST variant where temporal coherence is more critical.

Conclusion

We integrated Spectral Transfer Units into $\pi_{0.5}$'s action prediction head via a residual spectral filtering block. On LIBERO, STU-K4 converges to near-baseline loss with 2.8% parameter overhead and no wall-clock cost, demonstrating that spectral sequence models can be modularly integrated into pre-trained VLA models without destabilizing fine-tuning.

References

[1] Physical Intelligence. π_0: A Vision-Language-Action Flow Model for General Robot Control. *arXiv:2410.24164*, 2024.
[2] Agarwal, N., Hazan, E., et al. Spectral State Space Models. *arXiv:2312.06837*, 2023.
[3] Zhao, W., Chen, K., et al. STUZero: Spectral Transfer Units for Zero-Shot World Models. *EVAR Lab*, 2024.
[4] Liu, B., et al. LIBERO: Benchmarking Knowledge Transfer for Lifelong Robot Learning. *NeurIPS*, 2023.
[5] Physical Intelligence. OpenPI: Open-source framework for π_0 models. `https://github.com/Physical-Intelligence/openpi`, 2024.